

LAB GUIDE WITH SOLUTIONS

Lab Guide

Ten labs with full solutions

Elixir 1.20.4 · Erlang/OTP 29 · approx. 6 hours

Elixir Training Programme

Elixir · the language, the platform and the fundamentals

Sessions 1-2 · Instructor-led · Elixir 1.20.4 on Erlang/OTP 29

How to use this guide

Ten labs, in order, roughly six hours including discussion. Each lab states a goal, gives you the tasks, and then gives the full solution with the real output it produced on the training machine.

Work the tasks before reading the solution. The value is in the moment where your output does not match the guide - that is where the learning is.

#	Lab	Minutes	Covers
1	The toolchain	25	install, versions, four executables
2	IEx as a workbench	30	REPL, helpers, i/1, h/1, v/1
3	Hello world, three ways	30	script, elixirc, mix run
4	Take a .beam file apart	40	compiled format, chunks, docs
5	Watch the compiler work	35	AST, macro expansion
6	Create a mix project	40	mix new, structure, mix.exs, test
7	Add dependencies	30	Hex, deps.get, mix.lock, deps.tree
8	Measure every data type	45	flat_size, words, bytes
9	Collections and their cost	40	list, tuple, map representations
10	Strings, binaries, the 64-byte line	40	UTF-8, heap vs refc binaries

IMPORTANT Windows users: the launchers are `iex.bat` and `mix.bat`. In PowerShell or cmd, plain `iex` collides with PowerShell's own Invoke-Expression alias - always type `iex.bat`.

WATCH OUT Every number in this guide was measured on Elixir 1.20.4 / Erlang/OTP 29 with an 8-byte word. Your scheduler count and atom counts will differ; the sizes will not.

LAB 1 · 25 MINUTES · SETUP

The toolchain

GOAL Confirm you have a working Elixir installation and learn what each of the four executables is for.

Task 1.1 — Report your versions

Run the version command and read every field in the output. Write down your OTP release, your ERTS version and your scheduler count.

Solution

```
> elixir --version
Erlang/OTP 29 [erts-17.0.6] [source] [64-bit] [smp:12:12] [ds:12:12:10]
[async-threads:1] [jit:ns]

Elixir 1.20.4 (compiled with Erlang/OTP 29)
```

WHY smp:12:12 means twelve schedulers on twelve logical cores. jit:ns means the just-in-time compiler is active. Your numbers will differ - the shape will not.

Task 1.2 — Ask the VM directly

Get the same facts from inside the runtime rather than from the banner.

Solution

```
iex> :erlang.system_info(:otp_release)
~c"29"
iex> :erlang.system_info(:schedulers_online)
12
iex> :erlang.system_info(:wordsize)
8
iex> :erlang.system_info(:process_limit)
1048576
```

WHY These are Erlang functions called from Elixir with no wrapper. An Erlang module is just an atom, so `:erlang.system_info/1` is an ordinary function call. Note the OTP release comes back as a charlist - Erlang's string type.

Task 1.3 — Identify the four executables

Without running anything, say what each of these does. Then check yourself.

Command	What it does	Leaves behind
<code>elixir f.exs</code>	Compiles in memory and runs	Nothing
<code>elixirc f.ex</code>	Compiles to bytecode on disk	<code>Elixir.Module.beam</code>
<code>iex</code>	Interactive shell	Nothing
<code>mix TASK</code>	Build tool - runs a task	<code>_build/</code> , <code>deps/</code>

CHECKPOINT You can state your OTP release and scheduler count from memory, and you know which of the four commands writes a file.

LAB 2 · 30 MINUTES · REPL

IEx as a workbench

GOAL Stop treating the REPL as a calculator. Use it to read documentation, inspect types and recover results.

Task 2.1 — The result history

Evaluate three expressions, then recover the second result without retyping it.

Solution

```
iex(1)> 1 + 1
2
iex(2)> x = [1, 2, 3]
[1, 2, 3]
iex(3)> length(x)
3
iex(4)> v(2)
[1, 2, 3]
```

WHY `v/1` returns the value that line `n` produced. `v()` with no argument returns the last result. This is a genuine time-saver when an expression was expensive to compute.

Task 2.2 — Read the docs without leaving the shell

Get the documentation and the typespec for `Enum.map/2`.

Solution

```
iex> h Enum.map

def map(enumerable, fun)

@spec map(t(), (element() -> any())) :: list()

Returns a list where each element is the result of invoking fun
on each corresponding element of enumerable.
```

WHY `h/1` reads the Docs chunk out of the compiled `.beam` file on disk. Documentation is part of the compiled artefact, not a separate website - which is why this works offline and why it is always the version you actually have.

Task 2.3 — Ask what something really is

Use `i/1` on these five values and note the two surprises: `42`, `:ok`, `true`, `"hi"`, `~c"hi"`.

Solution

```
iex> i 42 -> Data type: Integer
iex> i :ok -> Data type: Atom
iex> i true -> Data type: Atom # surprise 1
iex> i "hi" -> Data type: BitString # surprise 2
iex> i ~c"hi" -> Data type: List
```

WHY `true` is not a Boolean type - it is the atom `:true`. `"hi"` is not a String type - it is a UTF-8 binary, and `BitString` is the underlying type. Both facts matter when you write guards.

Task 2.4 — Reload code without restarting

Create a module in a file, load it into the running shell, edit it, and reload it.

Solution

```
iex> c "greeter.ex"
[Greeter]
iex> Greeter.greet("world")
"Hello, world!"
# now edit greeter.ex, then:
iex> r Greeter
{:reloaded, [Greeter]}
```

WHY Inside a mix project use `recompile()` instead - it rebuilds everything that changed. The BEAM can hold two versions of a module at once, which is what makes this safe while processes are running.

CHECKPOINT You can read a function's docs, inspect a value's real type, and reload a module without restarting the shell.

LAB 3 · 30 MINUTES · FIRST PROGRAM

Hello world, three ways

GOAL See the three execution models - script, compiled module, mix project - and what each leaves on disk.

Task 3.1 — As a script

Create `hello.exs`, run it, and confirm nothing was written to disk.

Solution

```
# hello.exs
IO.puts("Hello!")

> elixir hello.exs
Hello!
> dir *.beam # nothing - the script compiled in memory
```

WHY The `.exs` extension is a convention, not a rule - elixir will run any file. What matters is that nothing is persisted, so scripts are the right tool for one-off work.

Task 3.2 — As a compiled module

Write a module to `greeter.ex`, compile it with `elixirc`, and find the file it produced.

Solution

```
defmodule Greeter do
  @moduledoc "A module small enough to read in bytecode form."
  @greeting "Hello"

  @doc "Greets someone by name."
  def greet(name) when is_binary(name), do: "#{@greeting}, #{name}!"

  def add(a, b), do: a + b
end

> elixirc greeter.ex
> dir
Elixir.Greeter.beam 2092 bytes
```

WHY The filename is `Elixir.Greeter.beam`, not `greeter.beam`. Every Elixir module name is prefixed with `Elixir`. because the module is really the atom `:"Elixir.Greeter"`. Keep this file - lab 4 takes it apart.

Task 3.3 — As a mix project

Create a project and run the same line through it.

Solution

```
> mix new greeter --module Greeter
* creating mix.exs
* creating lib/greeter.ex
* creating test/greeter_test.exs
...
> cd greeter
```

```
> mix run -e 'IO.puts("Hello!")'  
Hello!
```

WHY `mix run` compiles the project first, then evaluates your expression with the project loaded. This is the only one of the three you would ever ship.

CHECKPOINT You can explain which of the three leaves a `.beam` file on disk and why the filename starts with Elixir.

LAB 4 · 40 MINUTES · COMPILED FORMAT

Take a .beam file apart

GOAL Open the compiled artefact and read what the compiler actually stored. This is the lab that makes the compile pipeline concrete.

Task 4.1 — List the chunks

Using `:beam_lib.all_chunks/1`, list every chunk in `Elixir.Greeter.beam` with its size in bytes.

Solution

```
path = ~c"Elixir.Greeter.beam"
{:ok, mod, chunks} = :beam_lib.all_chunks(path)
for {id, data} <- chunks do
  IO.puts("#{id} #{byte_size(data)}")
end

AtU8 196 atom table (UTF-8)
Code 225 BEAM virtual-machine bytecode
StrT 8 string literal table
ImpT 40 imported functions
ExpT 64 exported functions
LitT 65 literal (constant) pool
LocT 16 local (private) functions
Attr 39 module attributes
CInf 187 compile info
Dbgi 529 debug info - the Elixir AST
Docs 273 @doc / @moduledoc
ExCk 259 Elixir type/export checker info
Line 35 line-number table for stack traces
Type 13 BEAM SSA type table
```

WHY Fourteen chunks for two functions. The bytecode - the Code chunk - is 225 bytes of a 2,092-byte file. Everything else is metadata: docs, debug info, type information, line numbers for stack traces.

Task 4.2 — Read the documentation back out

Use `Code.fetch_docs/1` to pull the docs out of the compiled file and prove that this is what `h/1` reads.

Solution

```
{:docs_v1, _, lang, fmt, moduledoc, _, docs} = Code.fetch_docs(Greeter)

language=elixir format=text/markdown
moduledoc="A module small enough to read in bytecode form."
function add/2 sig=["add(a, b)"] doc=false
function greet/1 sig=["greet(name)"] doc="Greets someone by name."
```

WHY `add/2` has `doc=false` because it carries no `@doc` attribute. Note that the parameter names survive compilation - that is how IEx can show you a signature.

Task 4.3 — Decompile the debug info

The `Dbgi` chunk holds the Elixir AST. Turn it back into Erlang and read what the BEAM compiler was actually given.

Solution

```
{:ok, {_, [debug_info: {:debug_info_v1, backend, data}]}} =
:beam_lib.chunks(path, [:debug_info])
{:ok, forms} = backend.debug_info(:erlang_v1, Greeter, data, [])
forms |> Enum.map(&:erl_pp.form/1) |> IO.chardata_to_string() |> IO.puts()

greet(_name@1) when is_binary(_name@1) ->
<<"Hello",
", ",
case _name@1 of
_1 when is_binary(_1) -> _1;
_1 -> 'Elixir.String.Chars':to_string(_1)
end/binary,
"! ">>.
```

WHY String interpolation is not concatenation. The compiler emits a binary construction, and it inserts a conditional: if the interpolated value is already a binary it is used directly, otherwise it goes through the `String.Chars` protocol. You can see the cost of interpolation, and why interpolating a binary is nearly free.

CHECKPOINT You can name three things stored in a `.beam` file besides bytecode, and you have seen your own source come back as Erlang.

LAB 5 · 35 MINUTES · AST AND MACROS

Watch the compiler work

GOAL See source become a data structure, and see a macro expand. This is the lab that explains why so much of Elixir is library code.

Task 5.1 — Source to AST

Turn the string "sum = 1 + 2 * 3" into an AST and identify the three parts of each node.

Solution

```
iex> Code.string_to_quoted("sum = 1 + 2 * 3")
{:ok,
 {:=, [line: 1],
 [{:sum, [line: 1], nil},
 {:+, [line: 1], [1, {:*, [line: 1], [2, 3]}]}]}
```

WHY Every node is {form, meta, args}. Precedence is already resolved - the multiplication node sits inside the addition node, so the tree decides evaluation order, not the text. This is ordinary Elixir data, which is exactly why macros can manipulate it.

Task 5.2 — Expand a macro by hand

Expand if/2 one step and read what it becomes.

Solution

```
iex> quote(do: if(x, do: 1, else: 2))
... |> Macro.expand_once(__ENV__)
... |> Macro.to_string()

case x do
x when :erlang.orelse(:erlang."!="(x, false), :erlang."!="(x, nil)) -> 2
_ -> 1
end
```

WHY if is a macro that expands to case. The guard spells out Elixir's truthiness rule exactly: only false and nil take the else branch. Everything else - including 0 and the empty string - is truthy.

Task 5.3 — Prove the pipe is a macro too

Expand a pipeline and show that it is a plain nested call.

Solution

```
iex> quote(do: [1,2,3] |> Enum.map(&(&1*2)) |> Enum.sum())
... |> Macro.expand_once(__ENV__) |> Macro.to_string()

"Enum.sum([1, 2, 3] |> Enum.map(&(&1 * 2)))"
```

WHY One expansion step rewrote the outermost pipe into a normal call. Expand repeatedly and the whole pipeline collapses to nested calls - the pipe has zero runtime cost.

Task 5.4 — Sugar is AST

Show that map syntax, keyword syntax and def are all ordinary AST nodes.

Solution

```
iex> Code.string_to_quoted!("%{a: 1}")
{: %{}, [line: 1], [a: 1]}
iex> Code.string_to_quoted!("[a: 1, b: 2]")
[a: 1, b: 2] # already a list of 2-tuples
iex> Code.string_to_quoted!("def f(x), do: x")
{: def, [line: 1], [{: f, [line: 1], [{: x, ...}]}, [do: {: x, ...}]]}
```

WHY def is not a keyword - it is a macro receiving two arguments, a function head and a keyword list. That is why you can write your own definition macros.

CHECKPOINT You can explain, with an example, why the pipe operator costs nothing at runtime.

LAB 6 · 40 MINUTES · MIX

Create a mix project

GOAL Build the project layout you will use for everything else, and understand every file mix generated.

Task 6.1 — Generate it

Solution

```
> mix new greeter --module Greeter
* creating README.md
* creating .formatter.exs
* creating .gitignore
* creating mix.exs
* creating lib
* creating lib/greeter.ex
* creating test
* creating test/test_helper.exs
* creating test/greeter_test.exs
```

WHY Nine files and no questions. Compare with the decision surface of a new Java or Node project: build tool, test framework, layout convention, formatter. Here all four are settled.

Task 6.2 — Account for every path

For each generated path, say what it is for and whether it belongs in version control.

Path	Commit?	What it is for
lib/	Yes	All compiled source. mix compiles every .ex under it.
test/	Yes	ExUnit finds files matching *_test.exs.
mix.exs	Yes	The project definition - config, deps, app.
mix.lock	Yes	Exact resolved versions and hashes.
.formatter.exs	Yes	Rules used by mix format.
_build/	No	Compiled output, one tree per MIX_ENV.
deps/	No	Fetches dependency sources - rebuildable.

Task 6.3 — Read mix.exs

Open mix.exs. It is a module with two public functions. Say what each key does.

Solution

```
defmodule Greeter.MixProject do
  use Mix.Project

  def project do
    [app: :greeter, # OTP application name, an atom
     version: "0.1.0", # semantic version, used by Hex
     elixir: "~> 1.20", # required compiler version
     deps: deps()] # a private function returning a list
  end
```

```
def application do
  [extra_applications: [:logger]] # started with your app
end
end
```

WHY `project/0` describes the build. `application/0` describes what the VM starts at boot. Both are ordinary functions - you can compute values in them, which is why `mix.exs` needs no separate templating language.

Task 6.4 — Compile, test, inspect the output

Solution

```
> mix compile
Compiling 1 file (.ex)
Generated greeter app

> mix test
..
Finished in 0.0X seconds
1 doctest, 1 test, 0 failures

_build/dev/lib/greeter/ebin/Elixir.Greeter.beam
_build/dev/lib/greeter/ebin/greeter.app
```

WHY `mix new` gives you a passing test suite from the start, including a doctest - a test extracted from the example in the module's `@doc`. Output lands under `_build/dev` because `MIX_ENV` defaults to `dev`; `mix test` uses `_build/test`.

Task 6.5 — Read the generated .app file

Solution

```
{application, greeter, [
  {modules, ['Elixir.Greeter']},
  {vsn, "0.1.0"},
  {applications, [kernel, stdlib, elixir, logger]}
]}.
```

WHY This is Erlang term syntax, generated for you from `mix.exs`. It is how the VM knows which modules belong to the application and which other applications must start first.

CHECKPOINT You can explain what `_build/dev` and `_build/test` are, and why `mix.lock` is committed but `deps/` is not.

LAB 7 · 30 MINUTES · MIX · HEX

Add dependencies

GOAL Add a runtime dependency and a dev-only tool, and see exactly what resolution produced.

Task 7.1 — Declare two dependencies

Add `jason` (JSON, needed at runtime) and `credo` (a linter, needed only while developing).

Solution

```
defp deps do
  [
    {:jason, "~> 1.4"},
    {:credo, "~> 1.7", only: [:dev, :test], runtime: false}
  ]
end
```

WHY `only:` restricts which MIX_ENV trees the dependency is compiled into. `runtime: false` means do not start it as an application - it is a tool you invoke, not a service your app needs. Together they keep dev tooling out of a production release.

Task 7.2 — Resolve and inspect

Solution

```
> mix deps.get
Resolving Hex dependencies...
Resolution completed in 0.234s
New:
bunt 1.0.0
credo 1.7.19
file_system 1.1.1
jason 1.4.5
* Getting jason (Hex package)
* Getting credo (Hex package)
* Getting bunt (Hex package)
* Getting file_system (Hex package)
```

WHY Two declared dependencies became four. `bunt` and `file_system` are transitive dependencies of `credo`. All four are pinned in `mix.lock` with a checksum.

Task 7.3 — See the tree, then the lock

Solution

```
> mix deps.tree
greeter
|-- jason ~> 1.4 (Hex package)
`-- credo ~> 1.7 (Hex package)
    |-- bunt (Hex package)
    `-- file_system (Hex package)

# mix.lock records the exact version and hash of all four
```

WHY Commit `mix.lock`. It is what makes a build reproducible - the `~> 1.4` constraint allows any 1.x, but the lock pins the one you actually tested against.

WATCH OUT `~> 1.4` means $\geq 1.4.0$ and $< 2.0.0$. `~> 1.4.2` means $\geq 1.4.2$ and $< 1.5.0$. The number of components decides where the ceiling sits.

CHECKPOINT You can explain the difference between `mix.exs` and `mix.lock`, and why a linter should be declared with `runtime: false`.

LAB 8 · 45 MINUTES · DATA TYPES

Measure every data type

GOAL Stop guessing what values cost. Measure them, and find the two boundaries that change representation.

Task 8.1 — Establish the unit

Before measuring anything, find out how big a word is on your machine.

Solution

```
iex> :erlang.system_info(:wordsize)
8 # bytes per word, 64-bit build
iex> :erts_debug.flat_size({1, 2, 3})
4 # words, including the header
# so {1, 2, 3} occupies 4 x 8 = 32 bytes on the process heap
```

WHY `flat_size/1` counts the words the term occupies, including everything reachable from it. A result of 0 means the value is an immediate - it fits inside the term word itself and costs no heap at all.

Task 8.2 — Find the small-integer boundary

Measure integers of growing size and find the exact point where an integer stops being free.

Solution

```
iex> :erts_debug.flat_size(42)
0
iex> :erts_debug.flat_size(Bitwise.<<<(1, 59) - 1)
0 # 2^59 - 1 is still immediate
iex> :erts_debug.flat_size(Bitwise.<<<(1, 59))
2 # 2^59 becomes a bignum
iex> :erts_debug.flat_size(2 ** 1000)
17 # 136 bytes
```

WHY On a 64-bit VM the boundary is $2^{59} - 1$ - four bits of the word are spent on tagging. Past it the integer moves to the heap as a bignum and grows linearly. Arithmetic never overflows - it just gets more expensive.

Task 8.3 — Measure the scalars

Measure a float, several atoms, a pid and a reference. Explain the two zeros.

Solution

```
3.14 flat = 2w 16 bytes
:ok flat = 0w 0 bytes
:some_very_long_atom_name_here flat = 0w 0 bytes
true flat = 0w 0 bytes
nil flat = 0w 0 bytes
self() flat = 0w 0 bytes
make_ref() flat = 3w 24 bytes
```

WHY Atom length is irrelevant to heap cost - the term is an index into a global atom table, so a 29-character atom costs the same as `:ok`. The atom text itself lives once, in that table, forever.

Task 8.4 — Prove that atoms are never collected

Count the atom table, create a thousand new atoms, force a garbage collection, and count again.

Solution

```
atom count before: 18992
# create 1000 atoms, then :erlang.garbage_collect()
atom count after: 19992

iex> :erlang.system_info(:atom_limit)
1048576
```

WHY The table only grows. Exhausting it halts the VM, which makes `String.to_atom/1` on user-controlled input a genuine denial-of-service vector. Use `String.to_existing_atom/1`, which raises rather than growing the table.

CHECKPOINT You can state the small-integer boundary, explain why atoms measure zero, and explain why dynamic atom creation is dangerous.

LAB 9 · 40 MINUTES · COLLECTIONS

Collections and their cost

GOAL Measure lists, tuples and maps, and find the point where a map silently changes representation.

Task 9.1 — Lists are exactly two words per cell

Solution

```
[ ] flat = 0w
[1] flat = 2w
[1, 2] flat = 4w
[1, 2, 3] flat = 6w
Enum.to_list(1..10) flat = 20w
Enum.to_list(1..100) flat = 200w
```

WHY One word for the value, one for the pointer to the rest. The relationship is exactly linear, which is why you can reason about list memory precisely.

Task 9.2 — Tuples are $n + 1$ words

Solution

```
{ } flat = 0w
{1} flat = 2w
{1, 2} flat = 3w
{1, 2, 3} flat = 4w
10-element tuple flat = 11w
```

WHY One header word holding the arity, plus one word per element, laid out contiguously. That contiguity is what makes `elem/2` constant time and `put_elem/3` $O(n)$.

Task 9.3 — Find the map representation switch

Measure maps from 1 key up to 100 keys. Find the size where the growth stops being smooth.

Solution

```
%{ } flat = 3w
%{a: 1} flat = 6w
%{a: 1, b: 2} flat = 8w
32 keys flat = 68w <- flatmap, the limit
33 keys flat = 123w <- hashmap, +55 words
100 keys flat = 364w
```

WHY Up to 32 keys a map is a flatmap: a sorted key array and a value array, compact and cache-friendly, with a scan for lookup. At 33 keys it becomes a hash array mapped trie - effectively $O(1)$ lookup, but each node costs. The syntax and the API do not change, so the only way to see this is to measure it.

NOTE A struct is a map with a fixed `__struct__` key, so a struct with fewer than 32 fields gets the cheap flat representation. Most structs do.

Task 9.4 — Feel the append trap

Build a 10,000-element list two ways - appending with ++, and prepending then reversing - and time both.

Solution

```
# quadratic: each ++ copies the whole left list
slow = fn n -> Enum.reduce(1..n, [], fn i, acc -> acc ++ [i] end) end

# linear: prepend is O(1), one reverse at the end
fast = fn n -> 1..n |> Enum.reduce([], fn i, acc -> [i | acc] end)
|> Enum.reverse() end

:timer.tc(fn -> slow.(10_000) end) |> elem(0) # microseconds
:timer.tc(fn -> fast.(10_000) end) |> elem(0)
```

WHY Exact timings vary by machine, but the ratio does not: the appending version is orders of magnitude slower and the gap widens with n . Build backwards, reverse once.

CHECKPOINT You can predict the size of a 50-element list and a 50-key map, and explain why the map answer is not simply proportional.

LAB 10 · 40 MINUTES · TEXT

Strings, binaries and the 64-byte line

GOAL Understand why strings are binaries, why byte length and character length differ, and where the runtime moves your data off-heap.

Task 10.1 — Bytes are not characters

Solution

```
iex> byte_size("héllö")
6
iex> String.length("héllö")
5
iex> String.codepoints("héllö")
["h", "é", "l", "l", "ö"]
```

WHY The accented character takes two bytes in UTF-8, so `byte_size` is 6 while there are 5 characters. `byte_size/1` is $O(1)$ - the length is in the header. `String.length/1` is $O(n)$ because it must walk and count graphemes.

Task 10.2 — Graphemes versus codepoints

Build a decomposed character - a plain e followed by a combining acute accent - and compare the three measurements.

Solution

```
iex> e = "e" <> <<0x301::utf8>>
iex> byte_size(e)
3
iex> String.codepoints(e)
["e", "◌́"] # two codepoints
iex> String.length(e)
1 # but one grapheme
```

WHY Three bytes, two codepoints, one grapheme - three different correct answers to "how long is this string?". The `String` module works in graphemes, which is almost always what a user means.

Task 10.3 — Find the 64-byte boundary

Measure binaries of 0, 2, 64, 65 bytes and 1 MB. Something counter-intuitive happens between 64 and 65.

Solution

```
"" flat = 2w
"hi" flat = 3w
String.duplicate("a", 64) flat = 10w <- heap binary, the limit
String.duplicate("a", 65) flat = 8w <- refc, smaller!
String.duplicate("a", 1MB) flat = 8w <- still 8 words
```

WHY Up to 64 bytes the data is copied onto the process heap. Past that the VM allocates the bytes once in a shared area and the process holds only an 8-word reference. That is why 65 bytes measures smaller than 64, and why a 1 MB binary measures the same as a 65-byte one - `flat_size` counts the reference, not the payload.

IMPORTANT This is the classic BEAM memory leak: take a small slice of a large reference-counted binary and the whole original stays alive, because the slice points into it. `:binary.copy/1` breaks the link when you only need the slice.

Task 10.4 — Charlists are a different thing entirely

Solution

```
iex> :erts_debug.flat_size("hi")
3 # binary
iex> :erts_debug.flat_size(~c"hi")
4 # list of 2 codepoints = 2 words each
iex> <<104, 105>>
"hi" # bytes ARE the string
iex> List.to_string(~c"hi")
"hi"
```

WHY A charlist is a list of integer codepoints, at 2 words per character against roughly one byte per character for a binary. You only need charlists when calling Erlang functions, which is where you will meet them.

Task 10.5 — Pattern match on bits

Unpack an RGB565 colour value - 5 bits red, 6 green, 5 blue - in a single match.

Solution

```
iex> <<r::5, g::6, b::5>> = <<0xF81F::16>>
iex> {r, g, b}
{31, 0, 31}

iex> is_binary(<<1::1>>)
false # not a whole number of bytes
iex> is_bitstring(<<1::1>>)
true
```

WHY No masks, no shifts. A binary is a bitstring whose bit count divides by 8; a bitstring has no such constraint. This is why the BEAM is unusually good at network protocol and codec work.

CHECKPOINT You can explain why a 65-byte binary measures smaller than a 64-byte one, and give three different correct answers to "how long is this string?".

Where to go next

These ten labs cover the two sessions delivered so far: the language and the platform, and the fundamentals of data. The natural next steps, in order of usefulness:

- **Pattern matching and control flow** — case, cond, with, and multi-clause functions. The single largest ergonomic difference from Java or JavaScript.
- **Enum, Stream and the pipeline style** — where eager and lazy differ, and why the pipe operator changes how you structure code.
- **Processes, GenServer and supervision** — the part of the platform that the whole design is built around.
- **Protocols and behaviours** — polymorphism without inheritance.
- **Mix in depth** — environments, releases, custom tasks, umbrella projects.

TIP Reproduce every measurement in this guide with two commands: `elixir verify/probe_sizes.exs` and `elixir verify/probe_lang.exs`. If a number here does not match your machine, trust your machine and tell the instructor.